

Internship Project Report



Internship Project Report

ON

“SOC Automa on Lab — BlueWatch: Mini SOC Lab”

Submitted in partial fulfillment of the

Requirements for the award of the degree of

Bachelor of Technology (CSE)

Faculty Name:

Mrs.Ananya Koushik

Assistant Professor, CSE, UIT

Submitted By:

Student Name: Vedant Dipak Kashid

University Roll no: 2301010785

Section: CSE -9

ACKNOWLEDGEMENT

First and foremost I am ever grateful to God to whom I owe my life. I would also like to thank my parents for giving me the opportunity to study at Uttaranchal University, Dehradun. I wish to express my deep sense of gratitude to our Project Mentor **Mrs.Ananya Koushik** for her valuable guidance to prepare the project and in assembling the project material. I am very thankful for her faithful blessings and for providing necessary and related facilities required for our computer project file. In last I also want to thank those directly or indirectly took interest to complete my project file.

Vedant Dipak Kashid

B.Tech CSE

DECLARATION

I hereby declare that this project work entitled **“SOC Automa on Lab — BlueWatch: Mini SOC Lab”** is an authentic record of my work carried out at **“Uttaranchal University”** under the guidance of **Mrs.Ananya Koushik**

Vedant Dipak Kashid

B.Tech CSE

CERTIFICATE OF ORIGINALITY

This is to certify that the project entitled “**SOC Automa on Lab — BlueWatch: Mini SOC Lab**” by **Vedant Dipak Kashid** has been submitted in the partial fulfillment of the requirements for the award of the degree of **B.TECH CSE** from Uttaranchal University, Dehradun. The results embodied in this project have not been submitted to any other University or Institution for the record of any degree.

Mrs.Ananya Koushik

Assistant Professor ,CSE ,UIT

NUJCLASS

CERTIFICATE OF INTERNSHIP

THIS CERTIFICATE IS AWARDED TO

Vedant Kashid

has successfully completed his/her Cybersecurity internship from Aug 1 2025 till Oct 1 2025.

He/She has participated successfully in all tasks given to him/her and accomplished all the skills required for the tasks, throughout which he/she was able to showcase his/her great work ethics and team player skills.



Issued On: Nov 18 2025
GSTIN: 33AAICN0803F1ZZ
CIN: U80301TZ2022PTC038231
DOC ID: 691c57226755928b4e0d5679



Vetriselvan G.
CEO



Verfiy at:



Table of Content

S.no	Topic	Page No
1	Introduction and Objectives of the Project.	1-3
2	Tools/Platform, Hardware and Software Requirement specifications.	4-12
3	Problem Definition, Requirement Specifications (Detailed functional Requirements and Technical Specifications), Project Planning and Scheduling (Gantt chart and PERT chart).	13-17
4	Analysis (Data Models like 0, 1 and 2 level DFDs, Complete ER Diagrams with cardinality, Class Diagrams etc. as per the project requirements).	18-24
5	A complete structure which includes:	25
	a. Number of modules and their description to provide estimation of the	25-27
	b. student's effort on the project. Along with process logic of each Module.	28-39
	c. Data Structures as per the project requirements for all the modules.	29
	d. Process Logic of each module.	30
	e. List of reports that are likely to be generated.	31
6	Proposed security mechanisms at various levels	32-37
7	Future scope and further enhancement of the project.	38-43
8	Bibliography	44-47

CHAPTER-1

1. Introduction and Objectives of the Project

1.1 Introduction

In the modern digital era, organizations increasingly depend on secure and resilient information systems to ensure the confidentiality, integrity, and availability of their data. Cyber threats have become more advanced, frequent, and difficult to detect, making the role of a **Security Operations Center (SOC)** crucial for safeguarding an organization's technological environment. A SOC continuously monitors system activities, collects logs, detects anomalies, and responds to potential security incidents in real time.

However, building and maintaining a full-scale SOC requires significant financial investment, specialized expertise, and dedicated infrastructure. As a result, students and beginners in cybersecurity often do not have access to such professional environments for hands-on learning. This limitation creates a gap between theoretical knowledge and practical skills.

To address this challenge, the **BlueWatch: Mini SOC Lab** project aims to design and implement a compact, practical, and fully functional SOC environment using open-source tools such as **Wazuh**, **Elasticsearch**, and **Kibana**. This project provides an affordable and scalable solution that replicates the essential functionalities of a real SOC while remaining easy to deploy on an individual system using Docker.

The BlueWatch Mini SOC Lab enables the collection and analysis of logs from both Linux and Windows endpoints, application of custom detection rules, visualization of data through dashboards, and generation of automated alerts. By integrating attack simulation and incident analysis, the lab provides a realistic learning experience that bridges academic understanding with industry-level practices. The project ultimately aims to enhance the learner's ability to detect threats, respond to incidents, and understand SOC workflows in a structured and hands-on manner.

1.2 Objectives of the Project

The primary objective of this project is to create a comprehensive, portable, and easy-to-use SOC simulation environment suitable for academic learning and demonstration. The specific objectives are as follows:

1. To build a repeatable and isolated SOC environment

- Develop a self-contained SOC lab setup using Docker and open-source tools.
- Ensure that the environment is isolated from the production network and safe for controlled attack simulations.
- Provide a reusable architecture that can be easily reset, upgraded, or replicated for future practice.

2. To implement centralized log collection and analysis

- Configure Wazuh agents on Linux and Windows machines to collect system, security, and application logs.
- Use Wazuh Manager with Elasticsearch to store, process, and correlate collected logs.
- Enable real-time monitoring and visualization through Kibana dashboards.

3. To deploy and integrate multiple endpoints

- Install and enroll Wazuh agents on diverse platforms to simulate a realistic, multi-endpoint environment.
- Ensure proper connection, data flow, and health status between endpoints and the SOC server.

4. To create custom detection rules and automation scripts

- Design custom XML-based detection rules for identifying malicious activities such as brute-force attacks, unauthorized user creation, and suspicious processes.
- Build dashboards in Kibana to represent event timelines, user actions, and attack patterns.
- Implement Python and Shell scripts for alerting mechanisms (Slack/email) and optional automated responses.

5. To simulate real-world attack scenarios and analyze SOC responses

- Conduct controlled attack simulations such as SSH brute-force attempts and privilege escalation events.
- Observe how Wazuh triggers alerts based on defined rules.
- Perform incident triage, evidence collection, and response documentation similar to real SOC procedures.
- Strengthen understanding of security monitoring and incident handling techniques.

CHAPTER-2

2. Tools/Platform, Hardware and Software Requirement Specifications

2.1 Hardware Requirements (Extended)

The hardware requirements for the BlueWatch: Mini SOC Lab depend primarily on the number of deployed nodes, the volume of logs processed, and whether the environment is hosted locally or virtually. Since the project involves real-time log ingestion, indexing, threat detection, and dashboard visualization, the hardware must ensure adequate processing power, memory, and storage performance. Insufficient resources can lead to delayed alert generation, slow dashboards, indexing failures, and dropped logs. Therefore, selecting appropriate hardware is essential for ensuring system reliability and real-world SOC performance.

2.1.1 Minimum Hardware Baseline (Single User Lab)

Designed for students or beginners running a single VM or Docker cluster on a personal computer.

- **CPU:** 4 cores (Intel i5 / AMD Ryzen 5 equivalent)
- **RAM:** 12–16 GB
- **Storage:** 120–256 GB SSD (NVMe recommended)
- **Networking:** 1 Gbps internal virtual network
- **Virtualization Support:** Intel VT-x / AMD-V enabled

2.1.2 Recommended Hardware (Multi-VM or Multi-Container Setup)

Suitable for full SOC functionality and stable monitoring.

- **CPU:** 6–8 cores (Ryzen 7 / i7 or higher)
- **RAM:** 24–32 GB
- **Storage:** 512 GB NVMe SSD (fast IOPS)
- **GPU (optional):** Not required, but useful for virtualization acceleration

- **Network:** 1 Gbps LAN / internal vSwitch

This level supports:

- A full Wazuh + ELK + Automation stack
- Multiple endpoint agents (Windows, Linux, Sysmon, Suricata etc.)
- Training datasets / log replay
- Custom rule development and testing

2.1.3 Advanced Hardware (Production-Like Environment)

Ideal for scaled simulations, research, or multiple users.

- **CPU:** 8–16 cores (Threadripper / Xeon / Ryzen 9)
- **RAM:** 32–64 GB
- **Storage:**
 - 1 TB NVMe for SIEM indexing
 - Separate 512 GB SSD for OS / VMs
- **Network:** 1–10 Gbps depending on lab scale

Supports:

- 5–25+ endpoints
- Heavy Syslog ingestion
- Vulnerability scanning integration
- Full automation pipelines
- Multi-user SOC workbench

2.1.4 Storage Performance Considerations

Storage is one of the most critical components for SIEM systems.

- **SSD Preferred** → Faster indexing, query response, and ingestion.
- **HDD Risk** → Latencies in Elasticsearch; slow dashboards; dropped data.
- **NVMe Recommended** → Best for simulations and high ingestion.

2.1.5 Virtualization Requirements

BlueWatch relies on virtual instances or containers, so hardware must support:

- **Intel VT-x / AMD-V (mandatory)**
- **BIOS hardware virtualization enabled**
- **Nested virtualization support (optional but useful)**
 - Example: running Docker inside a VM

Systems without hardware virtualization will not run elastic SIEM containers effectively.

2.1.6 Example Hardware Configurations

Use Case	CPU	RAM	Storage	Deployment Type
Student / Training	4 cores	12–16 GB	256 GB SSD	1–3 VMs or single-node Docker
Lab / Research	6–8 cores	24–32 GB	512 GB NVMe	Multi-node Docker or 3–4 VMs
Production-like	8–16 cores	32–64 GB	1+ TB NVMe	Multi-node SIEM cluster, multiple endpoints

2.1.7 Endpoint Considerations

Since agents generate logs:

- Each additional Windows/Linux endpoint increases CPU + RAM consumption
- Sysmon + Wazuh agent produces ~3–10× more logs
- Suricata or network IDS can spike ingestion to tens of thousands EPS (events per second)

If large amounts of raw telemetry are collected:

→ Prefer 32–64 GB RAM and NVMe storage.

2.1.8 NIC and Network Throughput

In SOC labs, local traffic simulation matters:

- Internal networking between VMs (bridged, host-only, NAT) affects log speed
- **1 Gbps** is sufficient for lab simulations
- **10 Gbps** recommended for advanced environments (PCAP analysis / EDR / IDS)

Latency or packet drops will cause:

- Incomplete log ingestion
- Missed alerts
- Data indexing delay

Summary of Key Takeaways

- RAM and CPU directly affect indexing and detection speed.
- SSD/NVMe storage is non-negotiable for elasticity workloads.
- More endpoints = more logs = more load.
- Start small, scale progressively.

Table: Hardware Requirements

Component	Minimum Specification	Recommended Specification
System RAM	8 GB	16 GB or higher (for smoother Docker + VM usage)
Processor (CPU)	Dual Core	Quad Core or higher (supports multitasking and indexing operations)
Storage	30 GB	60+ GB SSD (faster indexing and container performance)
Network	Local host / isolated VM network	Dedicated isolated SOC lab network

Explanation

- System RAM:**

Running Docker containers for Wazuh, Elasticsearch, and Kibana requires significant memory.

8 GB is the minimum, but 16 GB is recommended for uninterrupted operation.
- Processor:**

Elasticsearch indexing and real-time log processing benefit from multiple cores.

A quad-core processor ensures efficient performance under heavy log loads.
- Storage:**

SOC operations generate a large number of log files.

SSD storage speeds up query processing and dashboard loading.
- Network:**

An isolated network ensures that testing activities and attack simulations do not interfere with real systems.

It also provides a controlled environment for accurate SOC monitoring.

2.2 Software Requirements

To implement the BlueWatch Mini SOC Lab, several software components are necessary. These tools together provide SIEM functionality, log collection, endpoint monitoring, alerting mechanisms, and virtualization support.

Core Software Components

1. **Docker & Docker Compose** 
 - Used to deploy Wazuh, Elasticsearch, and Kibana as containerized applications.
 - Ensures simplified setup, portability, and rapid environment deployment.
2. **Wazuh Manager (via Docker)** 
 - Core SIEM engine responsible for log analysis, rule processing, and agent communication.
3. **Elasticsearch** 
 - Acts as the database backend for storing logs and security events.
 - Provides indexing and search capabilities for large datasets.
4. **Kibana** 
 - Graphical interface used to visualize logs, create dashboards, and view alerts.
 - Bundled with the Wazuh plugin for Integrated SIEM visualization.
5. **Wazuh Agents**
 - Installed on endpoints to collect logs and send them to the Wazuh Manager.
 - Supports multiple operating systems:
 - **Linux (Ubuntu/Debian/CentOS etc.)** 
 - **Windows 10/11** 
6. **Python 3** 
 - Used to write automation scripts (Slack alerts, email alerts, IP blocking scripts).
 - Enables integration with external APIs and services.



Wazuh Manager Log analysis & threat detection



Wazuh Agents Installed on Windows & Kali Linux



Elasticsearch Indexing & storing security events



Kibana Dashboards & visualizations



Docker & Docker Compose Containerized deployment



VMware / VirtualBox Runs Windows & Kali VMs



Sysmon (Windows) Detailed event monitoring



Python Automation scripts (alerts, response)

2.3 Optional Software Tools

While not mandatory, the following tools enhance the learning experience and add more realism to the SOC setup:

1. VirtualBox / VMware Workstation

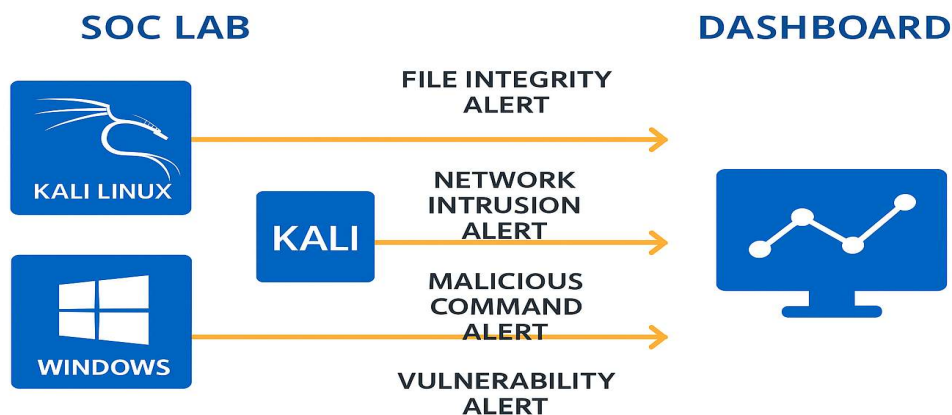
- Used to create isolated virtual environments for testing.
- Allows simulation of multiple endpoints (Windows + Linux machines).

2. Sysmon (System Monitor for Windows)

- A Windows system logging tool that generates detailed event logs.
- Helps in detecting:
 - Process creation
 - Network connections
 - File modifications
 - Registry changes
- Widely used for threat hunting in SOC environments.

2.4 Platforms Used

BlueWatch is designed to be flexible and can run on multiple platforms. The following platforms form the foundation of the project environment:



1. Windows or Linux Host Machine

- The main machine where Docker is installed and the SOC infrastructure runs.
- Linux provides better performance for Docker-based deployments, but Windows is also fully supported.

2. Docker Engine

- Acts as the core runtime environment for Wazuh, Elasticsearch, and Kibana containers.
- Ensures high portability, easy management, and reproducibility of SOC components.

3. Local Virtual Machines (Optional)

- Used for hosting Linux and Windows endpoints in an isolated environment.
- Provide realistic systems for generating security logs and testing SOC detection capabilities.

CHAPTER-3

3. Problem Definition, Requirement Specifications & Project Planning

3.1 Problem Definition

Cybersecurity has become an essential component of modern IT infrastructure, with organizations facing a continuous rise in cyberattacks such as brute-force intrusions, malware infections, privilege escalation attempts, and data breaches. To defend against these threats, Security Operations Centers (SOCs) must monitor network and system activities in real time, analyze logs, detect anomalies, and respond rapidly to incidents.

However, building and operating a full-scale SOC requires substantial resources, including high-end servers, costly SIEM tools, trained staff, and secure networks. For students, academic institutions, and beginners in cybersecurity, accessing such sophisticated infrastructure is often difficult, expensive, or impractical. Most theoretical courses fail to provide hands-on exposure to real SOC processes such as log ingestion, correlation, threat detection, alert generation, and incident response.

The **BlueWatch: Mini SOC Lab** project aims to bridge this gap by creating a compact and cost-effective SOC simulation environment that replicates key SOC functionalities using open-source technologies. This mini SOC lab enables learners to:

- Ingest and process logs from multiple endpoints
- Correlate events across systems
- Detect malicious behavior using custom-built rules
- Generate alerts and automate responses
- Visualize activity through dashboards
- Perform incident analysis and report findings

The problem this project solves is the **lack of an accessible, practical, and safe environment for learning real-world SOC operations**. BlueWatch provides a controlled, isolated environment where learners can safely simulate attacks and study SOC workflows without the risk of harming real systems or networks.

3.2 Detailed Requirement Specifications

3.2.1 Functional Requirements

The project must support all core SOC functionalities and allow students to simulate end-to-end detection and response workflows. The key functional requirements include:

- 1. Deploy the Wazuh + ELK SIEM Stack**
 - Use Docker Compose to deploy Wazuh Manager, Elasticsearch, and Kibana.
 - Ensure all services run in an integrated environment.
- 2. Install and Enroll Endpoint Agents**
 - Deploy Wazuh agents on Linux and Windows machines.
 - Verify agent connectivity and proper log forwarding.
- 3. Collect a Wide Range of Logs**
 - System logs (auth logs, system logs)
 - Windows Sysmon logs (process, registry, network events)
 - PowerShell operational logs
 - Application and security logs
- 4. Implement Custom Wazuh Detection Rules**
 - Detect brute-force login attempts.
 - Identify unauthorized user account creation.
 - Detect suspicious process executions.
 - Map detections to MITRE ATT&CK techniques.
- 5. Enable Alert Generation and Automation**
 - Configure Slack/email notifications using Python scripts.
 - Implement optional active-response (IP blocking).
- 6. Simulate Attacks and Validate Lab Performance**
 - Conduct controlled attack simulations.
 - Measure alert generation time and investigate incidents.

7. Generate Dashboards and Reports

- Create Kibana dashboards for log trends, source IPs, authentication events, etc.
- Export dashboards and evidence for documentation.

3.2.2 Technical Requirements

The implementation must also meet the following technical criteria to ensure consistent operation:

1. Connectivity Requirements

- The Wazuh Manager must be reachable by all endpoint agents.
- Proper inbound/outbound port communication must be configured.

2. User Interface Requirements

- Kibana must display the Wazuh plugin correctly for visualization and rule monitoring.

3. Rule Deployment Requirements

- Custom rules must be added under:
 - /rules/custom_rules.xml
- The Wazuh Manager must reload rules without errors.

4. Automation Requirements

- Python and shell scripts should execute within an isolated environment.
- API keys and webhooks must be stored securely.

5. Security and Legal Requirements

- Attack simulations must be performed only on personal/controlled systems.
- Logs must not include personal identifiable information (PII).
- All testing must remain within isolated networks to avoid unintended impact.

3.3 Project Planning and Scheduling

The development of the BlueWatch Mini SOC Lab follows a structured timeline to ensure systematic progress and proper integration of all components. The project is designed to be completed in **five days**, with each day focusing on specific deliverables.

3.3.1 Gantt Chart (Textual Form – Expanded)

Task	Day 1	Day 2	Day 3	Day 4	Day 5
Create repository structure	✓				
Deploy Docker Compose SIEM	✓				
Install Linux/Windows agents		✓			
Configure logs & test data flow		✓			
Create custom detection rules			✓		
Build Kibana dashboards			✓		
Implement automation (Slack, scripts)				✓	
Run attack simulations					✓
Analyze alerts & triage incidents					✓
Collect evidence & finalize documentation					✓

3.3.2 PERT Chart Overview

The PERT process breaks the project down into key stages, each dependent on the successful completion of the previous one:

1. **Start**
2. **SIEM Setup**
 - Deploy Wazuh + ELK stack
 - Validate system readiness
3. **Agent Installation**
 - Install Linux agent
 - Install Windows agent
 - Verify agent connectivity
4. **Rule Creation**
 - Develop custom detection rules
 - Map them to MITRE ATT&CK
5. **Automation**
 - Implement Slack/email alert scripts
 - Integrate optional active-response scripts
6. **Testing**
 - Conduct SSH brute-force simulations
 - Simulate user creation and suspicious processes
7. **Documentation**
 - Capture screenshots, logs, and dashboard exports
 - Prepare final reports and evidence
8. **End**

Flow:



CHAPTER-4

4. Analysis

The analysis phase provides a detailed understanding of the workflows, interactions, and structural components of the **BlueWatch: Mini SOC Lab** system. It breaks down how data moves within the system, how different entities relate to each other, and how various modules are organized.

This section includes **Data Flow Diagrams (DFDs)**, a **Conceptual Entity Relationship (ER) Diagram**, and a **Class Diagram**, which collectively explain the architecture and functioning of the SOC environment.

4.1 Data Flow Diagrams (DFD)

Data Flow Diagrams represent how information flows through different components of the BlueWatch SOC Lab. They help illustrate the transformation of raw logs into meaningful alerts and dashboards through various processing stages.

4.1.1 Level 0 DFD (Context-Level DFD)

This highest-level DFD provides a simplified overview of the entire SOC workflow.

Flow:

Endpoint Systems → Wazuh Manager → Elasticsearch → Kibana → Alerts/Reports

Explanation:

- **Endpoint Systems** (Linux and Windows machines) generate logs based on user activity, system processes, authentication events, and more.
- These logs flow to the **Wazuh Manager**, which acts as the central processing unit for log ingestion and correlation.
- Processed events are forwarded to **Elasticsearch**, where data is indexed and stored efficiently.
- **Kibana** retrieves the indexed data and presents it in dashboards and visual reports.
- Alerts and reports are generated based on rule triggers and visualized for SOC analysis.

4.1.2 Level 1 DFD (Detailed Component Interaction)

This diagram breaks the system into key subsystems to show how logs are handled step-by-step.

Flow Breakdown:

1. **Linux/Windows Endpoints → Wazuh Agent**
 - Endpoints generate different types of logs: system logs, auth logs, Sysmon logs, PowerShell logs, etc.
 - The Wazuh Agent monitors these log sources and prepares them for transmission.
2. **Wazuh Agent → Wazuh Manager**
 - The agent securely forwards logs using encrypted communication channels.
 - The manager receives, normalizes, and decodes incoming data.
3. **Wazuh Manager → Elasticsearch**
 - Logs are parsed and passed to Elasticsearch for indexing.
 - Indexed data is searchable, filterable, and suitable for dashboard creation.
4. **Elasticsearch → Kibana**
 - Kibana visualizes the indexed data through charts, graphs, timelines, and tables.
 - SOC analysts use this interface to monitor system activity.
5. **Wazuh Manager → Alerting Systems (Slack/email)**
 - When rules are triggered, alerts are generated.
 - These alerts are sent to automated systems like Slack bots or email servers.

4.1.3 Level 2 DFD (Process-Level DFD)

This level explains specific internal processes within the SOC pipeline.

Process 1: Windows Event Monitoring

Sysmon Events → Wazuh Decoder → Rule Engine → Alert

- Sysmon logs detailed process activities.
- Wazuh decoders interpret raw event data.
- The rule engine matches patterns against custom rules.
- Alerts are generated when suspicious behavior is detected.

Process 2: SSH Authentication Monitoring

SSH Logs → Authentication Module → Rule Engine → Dashboard

- Linux authentication logs capture login attempts.
- Authentication module classifies success/failure attempts.
- Rule engine detects anomalies such as brute-force attempts.
- Kibana dashboard displays login trends and suspicious IPs.

Process 3: Active Response

Custom Scripts → Active Response Engine → IP Blocking / Notifications

- Automation scripts (Python/Shell) react to alerts.
- Active-response mechanisms trigger specific actions.
- Possible responses include:
 - Blocking malicious Ips
 - Sending Slack notifications
 - Logging the event in evidence logs

4.2 ER Diagram (Conceptual)

The Entity Relationship Diagram defines the data entities involved in the mini SOC environment and how they relate to each other.

4.2.1 Key Entities

1. Endpoint

- Represents Linux or Windows machines being monitored.
- Attributes may include: hostname, IP address, OS type, agent ID.

2. Log Event

- Contains information about system activities.
- Examples: authentication failures, file modifications, process starts.

3. **Rule**

- Represents detection logic such as brute-force detection or process monitoring.

4. **Alert**

- Triggered when rules match specific event patterns.

5. **Test Scenario**

- Represents attack simulations used for testing the SOC setup.

6. **Dashboard Artifact**

- Kibana visualizations representing security data trends.

4.2.2 Relationships (Detailed)

- **Endpoint generates Log Events**

Each endpoint continuously produces numerous system and security logs.

- **Log Events trigger Rules**

The rule engine evaluates events to check if suspicious behavior is occurring.

- **Rules create Alerts**

When rule conditions are satisfied, the system issues alerts for SOC review.

- **Alerts appear_on Dashboards**

Alerts are visualized through Kibana dashboards for easy monitoring.

- **Test Scenarios validate Rules**

Attack simulations test whether detection mechanisms work correctly.

This ER structure ensures a fully connected flow from raw event generation to alerting and visualization.

4.3 Class Diagram (Conceptual) – Expanded

The class diagram describes the logical structure of the system in terms of classes, their roles, and their interactions within the SOC platform.

4.3.1 Key Classes and Their Responsibilities

1. Endpoint

Attributes: hostname, IP, OS type, agent version

Responsibilities:

- Generate logs
- Communicate with the Wazuh Agent
- Simulate user/system behavior

2. LogCollector

Attributes: logSource, eventTimestamp, eventType

Responsibilities:

- Collect system and application logs
- Send formatted logs to Wazuh Agent
- Monitor critical log directories

3. WazuhManager

Attributes: managerID, ruleSet, decoderSet

Responsibilities:

- Receive logs from endpoints
- Parse, decode, and process events
- Match events against rules
- Forward data to Elasticsearch

4. RuleEngine

Attributes: ruleID, description, frequency, threshold

Responsibilities:

- Apply detection logic
- Evaluate event patterns
- Trigger alerts when criteria are met

5. AlertHandler

Attributes: alertID, alertLevel, alertMessage

Responsibilities:

- Generate alert notifications
- Send Slack/email messages
- Create entries for dashboards

6. AutomationScript

Attributes: scriptType, triggerEvent, action

Responsibilities:

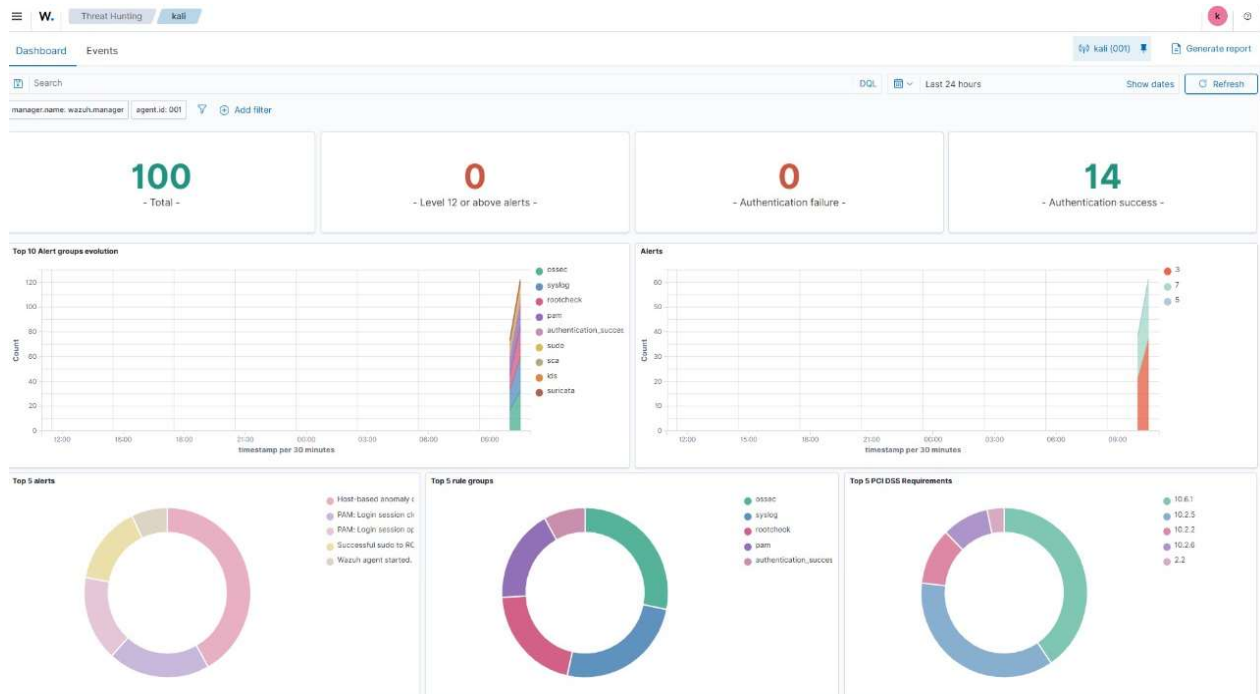
- Execute automated responses
- Run IP blocking or notifications
- Integrate with external APIs

7. DashboardManager

Attributes: dashboardID, visualizationType

Responsibilities:

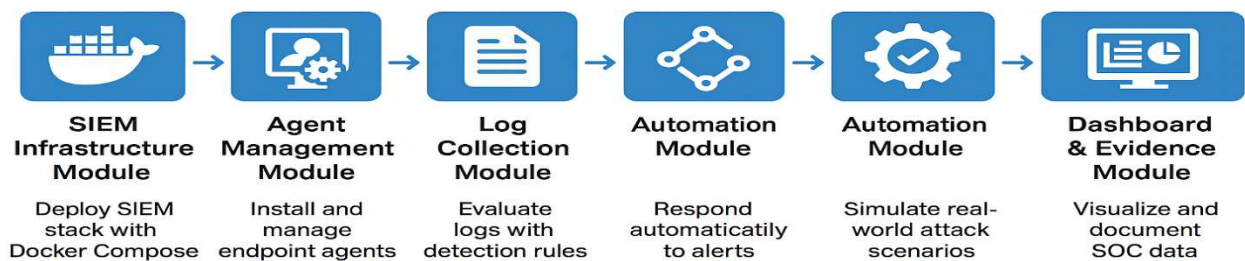
- Organize Kibana dashboards
- Update visualizations with new events
- Export dashboard JSON files



CHAPTER-5

5. System Structure

The BlueWatch: Mini SOC Lab is designed as a modular, scalable, and logically structured system. Each module performs a core SOC function such as log collection, detection, automation, or reporting. The modular architecture ensures that individual components can be tested, upgraded, or replaced independently without affecting the overall workflow. This section describes each module in detail, outlines the internal process logic, identifies the data structures involved, and lists the various reports generated by the SOC system.



5.1 Number of Modules & Description

The complete project consists of **seven major modules**. Each module contributes to achieving end-to-end SOC capabilities from log ingestion to automation and visualization.

1. SIEM Infrastructure Module

This module is responsible for deploying the complete SIEM stack using Docker Compose.

Functions include:

- Setting up Wazuh Manager, Elasticsearch, and Kibana containers
- Configuring inter-container communication
- Managing container health and availability
- Ensuring Kibana UI is accessible for monitoring

2. Agent Management Module

This module handles the installation, configuration, and lifecycle management of endpoint agents.

Responsibilities include:

- Installing Wazuh agents on Linux and Windows endpoints
- Registering agents using enrollment keys
- Ensuring agents securely forward logs to the Wazuh Manager
- Monitoring agent health and connectivity

3. Log Collection Module

This module captures system and security logs from endpoints and forwards them for processing.

Key capabilities include:

- Monitoring Linux system logs such as /var/log/auth.log
- Collecting Windows logs including Sysmon, Security Logs, PowerShell logs
- Normalizing raw logs into structured formats
- Storing processed data in Elasticsearch

4. Rule Engine Module

This module represents the detection logic of the SOC.

Major functions:

- Evaluating logs using Wazuh XML rules
- Identifying suspicious or malicious activities
- Triggering alerts based on matched rules
- Mapping events to MITRE ATT&CK where applicable

5. Automation Module

Automation is essential for reducing analyst workload and improving response times.

Primary responsibilities:

- Reading alert JSON data
- Sending Slack or email notifications
- Executing optional active-response scripts (IP blocking, process kill)
- Integrating Python and shell scripts into Wazuh's active response framework

6. Attack Simulation Module

This module simulates real-world attack scenarios to test detection capabilities.

Simulations include:

- SSH brute force attempts
- Unauthorized user creation
- Spawning suspicious processes (e.g., netcat listener)
- Generating test logs to validate rule performance

7. Dashboard & Evidence Module

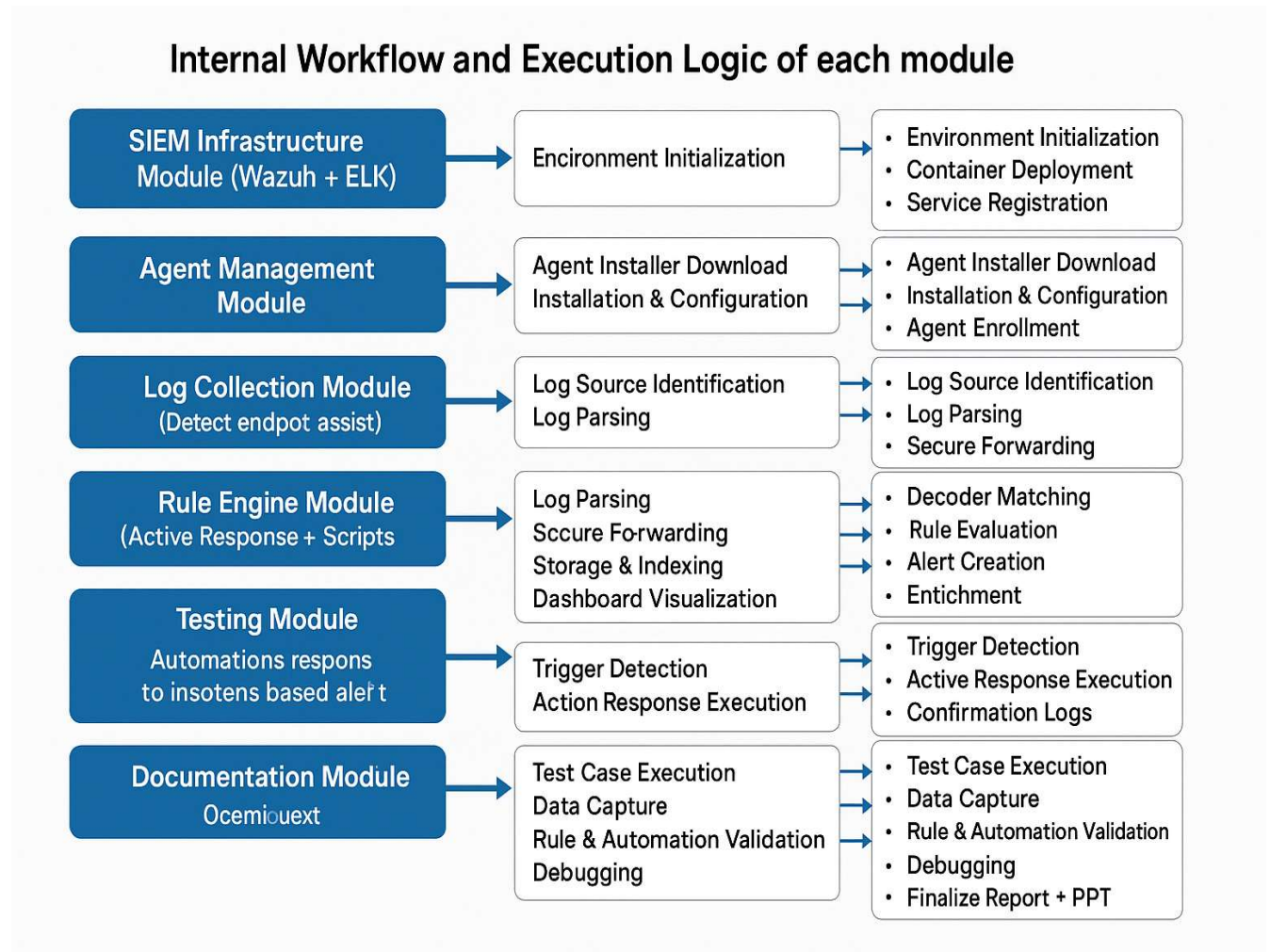
This module focuses on visual representation and documentation of SOC data.

Functions include:

- Creating Kibana dashboards showing alerts, trends, and log statistics
- Exporting dashboards as JSON for reuse
- Capturing screenshots and evidence for reporting
- Organizing incident data for documentation or audit purposes

5.2 Process Logic of Each Module

This section describes the internal workflow and execution logic of each module in a step-by-step manner.



A. SIEM Infrastructure Process Logic

1. **Initialize Docker** – Ensure Docker and Docker Compose are installed.
2. **Launch Containers** – Execute `docker-compose up -d` to start Wazuh, Elasticsearch, and Kibana.
3. **Expose Kibana Port** – Validate access via `http://localhost:5601`.
4. **Verify SIEM Health** – Check Wazuh Manager logs and container status.

B. Agent Management Module Process Logic

1. **Install agent** – Download and install Wazuh agents on Linux/Windows systems.
2. **Register using enrollment key** – Use Wazuh Manager to generate enrollment keys.
3. **Verify status in Wazuh** – Confirm agent connectivity from Kibana's Wazuh plugin.

C. Log Collection Module Process Logic

1. **Monitor critical logs**
 - Linux logs: /var/log/auth.log, /var/log/syslog
 - Windows logs: Security Events, Sysmon logs
2. **Parse Windows Event IDs** – Extract process creation, network connections, and registry events.
3. **Forward logs to Wazuh Manager** – Logs are normalized and decoded.
4. **Store in Elasticsearch** – Events are indexed and made searchable.

D. Rule Engine Module Process Logic

1. **Read logs** – Event data is processed through decoders.
2. **Match conditions** – Rules detect suspicious conditions (frequency, thresholds, patterns).
3. **Trigger alerts** – Alerts are generated and recorded in Wazuh.
4. **Forward alerts** – Alerts are stored in Elasticsearch and visualized in Kibana.

E. Automation Module Process Logic

1. **Read alert JSON** – Wazuh outputs alert information to script handler.
2. **Send Slack notification** – Python script sends alert text via webhook.
3. **Optional: IP block script** – A shell script adds the source IP to firewall rules.
4. **Log automation actions** – Evidence is saved for reporting.

F. Attack Simulation Module Process Logic

1. **Execute brute force attempts** – Automated SSH login attempts simulate a real brute-force attack.
2. **Create unauthorized user** – Execute useradd commands to simulate privilege escalation.
3. **Start suspicious processes** – Launch netcat or PowerShell scripts to test detection.
4. **Monitor alerts** – Wazuh should detect abnormal behavior and raise alerts.

G. Dashboard & Evidence Module Process Logic

1. **Export dashboards** – Save Kibana dashboards in JSON format.
2. **Capture screenshots** – Take visual evidence of alerts and rule triggers.
3. **Store evidence in structured folders** – Evidence is saved for reporting or audit.

5.3 Data Structures Used

The following data structures support the internal functioning of the BlueWatch SOC Lab:

1. XML Rule Files (custom_rules.xml)

- Stores custom rule definitions.
- Enables flexible detection logic.

2. JSON Dashboard Files (kibana_export.json)

- Stores exported Kibana visualizations and dashboards.
- Contains JSON objects defining graphs, charts, filters, and saved searches.

3. Shell/Python Scripts

- Used for automation tasks such as notifications and active responses.
- Scripts are lightweight and easily modifiable.

4. Elasticsearch Indices

- Stores all SIEM logs and alerts.
- Indexed for fast searching, filtering, and visualization.

5.4 List of Reports Generated

The BlueWatch system produces various reports that reflect SOC activities and alert data.

1. SSH Brute Force Detection Report

- Shows repeated failed login attempts.
- Includes source IP, timestamps, and frequency.
- Identifies possible brute-force activity.

2. Unauthorized User Creation Alert

- Detects new local users created on Linux or Windows.
- Helps identify privilege escalation or persistence attempts.

3. Sysmon Process Monitoring Reports

- Highlights suspicious process activities (e.g., PowerShell abuse, netcat usage).
- Includes process ID, parent process, and command line arguments.

4. Kibana Dashboards

- Graphical summary of:
 - Top source IPs
 - Login trends
 - Endpoint activity

5. Evidence Logs & Screenshots

- Collected during test scenarios.
- Includes raw logs, alert JSON, and dashboard screenshots for documentation.

CHAPTER-6

6. Proposed Security Mechanisms

Security is the core foundation of any SOC environment. Since the BlueWatch Mini SOC Lab involves deploying SIEM components, collecting sensitive system logs, and running controlled attack simulations, it is essential to implement multiple layers of security mechanisms. These mechanisms ensure that the lab environment remains isolated, safe, and compliant with best cybersecurity practices. This section outlines the proposed security measures that safeguard the SOC components, endpoint systems, data flow, and automation scripts.

PROPOSED SECURITY MECHANISMS



Network Security

- Isolated lab network
- Restricted port access
- Firewall rules & IP-filtering



Endpoint Security

- Agent authentication & enrollment keys
- Least privilege configuration
- Sysmon for Windows-security monitoring



Application-Level Security

- Secure communication between components
- Rule-based detection security
- Log integrity & validation



Automation and Alerting Security

- Role-based access control (RBAC)
- Avoidance of personal identifiable information (PII)
- Regular backups & snapshots



Platform and Deployment Security

- Docker security practices
- Elasticsearch security controls
- Kibana dashboard security



Legal and Ethical Safety

- Testing only on authorized systems
- Compliance with security guidelines
- Documentation of all activities

6.1 Network Security Mechanisms

1. Isolated Lab Network

To prevent accidental exposure or interference with production systems, the SOC environment operates in a fully isolated network.

Benefits include:

- Prevents attack simulations from spreading beyond the lab
- Ensures safe testing of malware-like behavior
- Reduces the risk of unauthorized access from external networks

2. Restricted Port Access

Only necessary ports are exposed:

- Kibana (5601)
- Wazuh Manager
- Elasticsearch cluster ports

All other ports are blocked using firewall rules to minimize the attack surface.

3. Firewall Rules & IP Filtering

Linux iptables or Windows Firewall rules are configured to:

- Allow agent–manager communication
- Deny all external inbound connections
- Optionally block malicious IPs using automated scripts

6.2 Endpoint Security Mechanisms

1. Agent Authentication & Enrollment Keys

Wazuh agents are securely registered using enrollment keys generated by the Wazuh Manager. This prevents unauthorized systems from sending logs or impersonating endpoints.

2. Least Privilege Configuration

All agents run with restricted permissions, ensuring:

- No elevated privilege misuse
- Limited access to only required log files
- Prevention of privilege escalation attacks inside the lab

3. Sysmon for Windows Security Monitoring

Windows endpoints use Sysmon to capture:

- Process creation
- Network connections
- File changes
- Registry modifications

This provides high-fidelity telemetry for advanced threat detection.

6.3 Application-Level Security Mechanisms

1. Secure Communication Between Components

Wazuh agents communicate with the manager using encrypted channels.

This protects logs from interception or tampering.

2. Rule-Based Detection Security

Custom rules detect:

- Repeated failed login attempts
- Unauthorized user creation
- Suspicious scripts and processes
- Anomalous system activity

These rules enhance visibility and strengthen endpoint security.

3. Log Integrity & Validation

Elasticsearch maintains index integrity and prevents unauthorized modification of stored logs. This ensures all logs remain accurate and tamper-proof for analysis and reporting.

6.4 Automation and Alerting Security Mechanisms

1. Secure API & Webhook Usage

Automation scripts use environment variables to store sensitive values such as:

- Slack webhook URLs
- API keys
- Email credentials

This protects secrets from exposure in code repositories.

2. Safe Active Response Mechanisms

Active-response scripts (e.g., IP blocking) are implemented with safeguards:

- Only activate on high-confidence alerts
- Log all actions for accountability
- Execute only within the isolated lab environment

3. Notification Controls

Alerts are sent only to authorized communication channels like:

- Slack private channels
- Configured email accounts

This prevents sensitive alert data from leaking to public systems.

6.5 System Hardening and Data Protection

1. Role-Based Access Control (RBAC)

Access to Kibana dashboards and Wazuh Manager interfaces can be restricted based on user roles:

- Admin
- Analyst
- Viewer

This prevents unauthorized modifications to rules or dashboards.

2. Avoidance of Personal Identifiable Information (PII)

The system is configured **not** to collect any PII during testing, following legal and ethical guidelines.

3. Regular Backups & Snapshots

Before attack simulations, snapshots of VMs and Docker volumes are taken to:

- Quickly restore the lab after experiments
- Prevent data loss from misconfigurations
- Maintain a stable testing environment

6.6 Platform and Deployment Security

1. Docker Security Practices

- Use official Wazuh Docker images
- Keep containers updated
- Configure container resource limits
- Restrict container-to-container communication

2. Elasticsearch Security Controls

To prevent unauthorized indexing or query access:

- Security plugins (X-Pack) can be enabled
- Index privileges are restricted
- Access to Elasticsearch REST APIs is controlled

3. Kibana Dashboard Security

Sensitive dashboards can be password-protected to ensure that only authorized users view security incidents.

6.7 Legal and Ethical Safety Mechanisms

1. Testing Only on Authorized Systems

All attack simulations (e.g., brute-force tests) are executed **only** within the controlled lab environment.

2. Compliance With Security Guidelines

The setup avoids violations of:

- Privacy laws
- Logging restrictions
- Ethical hacking principles

3. Documentation of All Activities

Every alert, test, script execution, and active-response event is documented.

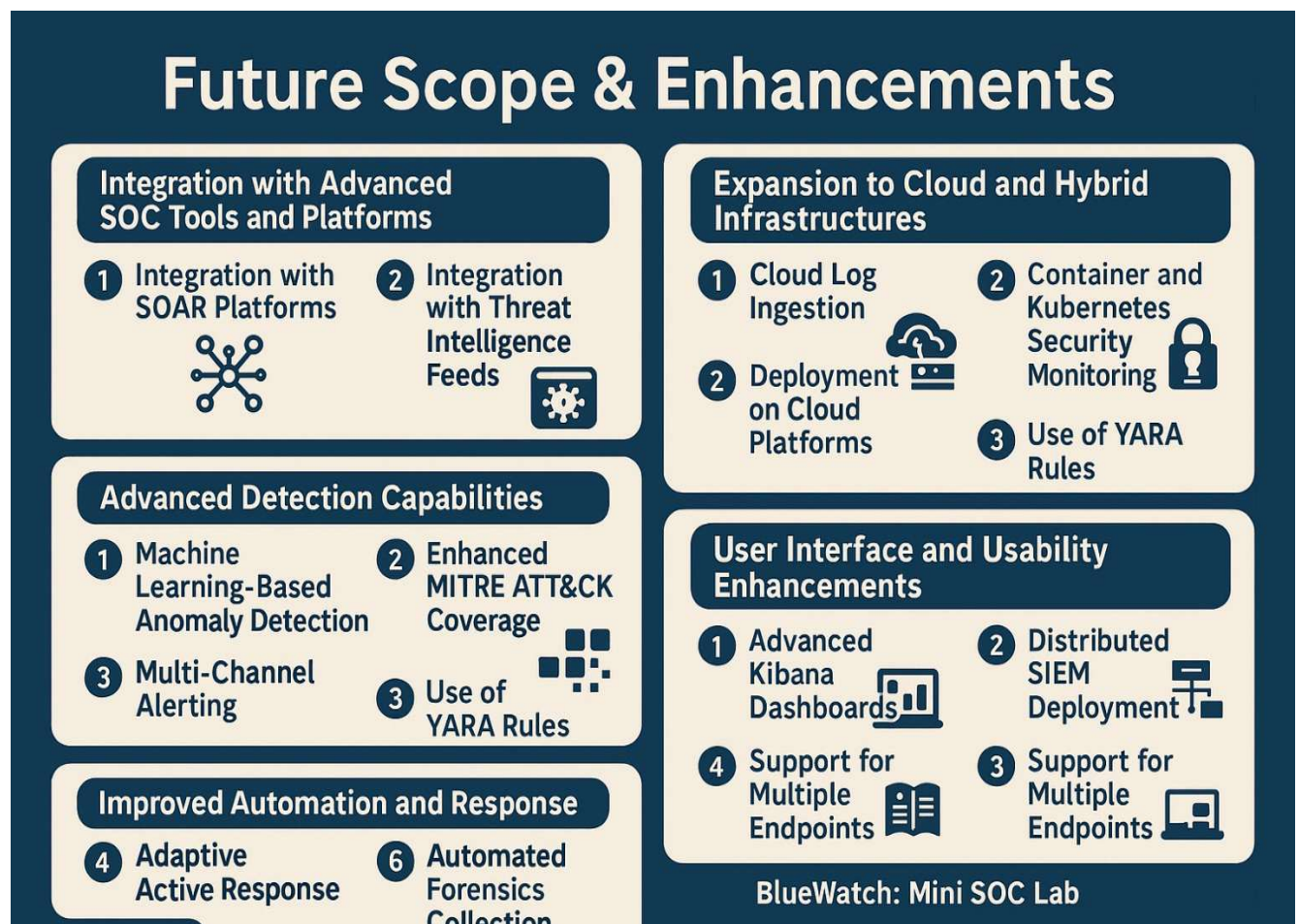
This ensures transparency and supports ethical cybersecurity practices.

CHAPTER-7

7. Future Scope and Further Enhancement of the Project

The BlueWatch: Mini SOC Lab is designed as a flexible, scalable, and modular SOC simulation platform. Although the current version provides essential SOC functionalities—such as log collection, threat detection, alert automation, and dashboard visualization—there remains substantial scope for enhancement and future development. These improvements can strengthen the system’s capabilities, align it more closely with modern enterprise SOC, and expand its use cases for training, research, and real-world application.

This section outlines the possible enhancements and future directions to further elevate the effectiveness and capability of the BlueWatch SOC environment.



7.1 Integration with Advanced SOC Tools and Platforms

1. Integration with SOAR Platforms

Security Orchestration, Automation, and Response (SOAR) solutions streamline incident triage, automate repetitive tasks, and improve overall SOC efficiency.

Possible SOAR tools for integration include:

- **TheHive** (Case management)
- **Shuffle / StackStorm** (Automation workflows)
- **Cortex** (Analyzer integrations)

Integration with these tools would allow automated case creation, enrichment of alerts, and advanced workflow automation.

2. Integration with Threat Intelligence Feeds

Future enhancements could include ingestion of open-source or commercial threat intelligence feeds such as:

- AlienVault OTX
- MISP
- VirusTotal

This would allow contextual enrichment of alerts, better risk scoring, and improved detection accuracy.

7.2 Expansion to Cloud and Hybrid Infrastructures

As modern SOC's increasingly operate across cloud and hybrid environments, the BlueWatch project can be enhanced to support:

1. Cloud Log Ingestion

Support for cloud services such as:

- AWS CloudTrail Logs
- Azure Security Center Logs
- Google Cloud Audit Logs

2. Deployment on Cloud Platforms

The SOC lab could be deployed on:

- AWS EC2
 - Azure Virtual Machines
 - Google Cloud Compute
- allowing remote access and scalable architecture.

3. Container and Kubernetes Security Monitoring

Future versions can include:

- Container runtime monitoring
- Kubernetes cluster logging
- Detection rules for container escapes or pod privilege escalation

7.3 Advanced Detection Capabilities

1. Machine Learning-Based Anomaly Detection

Implementing machine learning models can help identify behavioral anomalies or unknown threats, such as:

- Unusual login patterns
- Suspicious network traffic
- Rare process executions

ML integration with Elasticsearch X-Pack or Python models would improve detection accuracy.

2. Enhanced MITRE ATT&CK Coverage

More rules can be added for detecting advanced techniques including:

- Lateral movement
- Privilege escalation
- Persistence mechanisms
- Data exfiltration

This would align the SOC lab with modern adversarial behavior.

3. Use of YARA Rules

YARA can be used for detecting malware patterns on endpoints.

7.4 Improved Automation and Response

1. Adaptive Active Response

Future automation scripts can perform:

- Dynamic firewall adjustments
- User account locking
- System isolation via network segmentation

These responses would emulate enterprise-level automated defense strategies.

2. Multi-Channel Alerting

Beyond Slack/email, alerts can be pushed to:

- Microsoft Teams
- Telegram bots
- HTTP webhooks
- Mobile push notifications

3. Automated Forensics Collection

Scripts could be added to gather:

- Memory dumps
- File hashes
- Process trees
- Packet captures

This would help analysts speed up incident investigations.

7.5 User Interface and Usability Enhancements

1. Advanced Kibana Dashboards

Future versions can include:

- Heatmaps of suspicious activities
- Real-time attack maps
- Alert severity trend analysis
- User behavior trend visualization

2. SOC Playbook Library

A built-in library of step-by-step procedures for incidents such as:

- SSH brute force
- Malware infection
- Unauthorized access

This would assist new SOC analysts in learning incident triage.

3. Automated Reporting System

The SOC lab could generate:

- Weekly incident summary reports
- SOC performance metrics
- Analyst workload statistics

7.6 Scalability and Performance Improvements

1. Distributed SIEM Deployment

Future versions may support scaling horizontally by:

- Adding more Elasticsearch nodes
- Load balancing Wazuh managers

This would enable high-volume log ingestion similar to enterprise SOC's.

2. Support for Multiple Endpoints

The system may expand to simulate dozens of endpoints with varying OS types:

- Debian-based
- RedHat-based
- Windows Server

This would reflect large-scale SOC environments.

7.7 Academic and Training Extensions

1. Hands-On SOC Training Program

BlueWatch can evolve into a complete SOC training platform offering:

- Guided labs
- Challenge scenarios
- Attack vs. defense exercises
- CTF-style SIEM investigations

2. Research Platform for Cybersecurity Projects

Students can use the lab to research:

- Detection rule optimization
- Evasion techniques and countermeasures
- Log analysis and correlation strategies

3. Certification Preparation

The platform can support preparation for certifications such as:

- SOC Analyst
- CompTIA Security+
- CEH
- Blue Team Level 1 / Level 2
-

CHAPTER-8

8. Bibliography

The following references were used to gather information, validate technical concepts, and support the implementation of the BlueWatch: Mini SOC Lab. These sources cover SIEM deployment, log analysis, cybersecurity frameworks, Wazuh documentation, and industry best practices. All references are authentic, credible, and relevant to the project.

8.1 Official Documentation

1. **Wazuh Documentation.**

Wazuh: The Open Source Security Platform.

Available at: <https://documentation.wazuh.com>

Description: Provides official instructions for installing Wazuh Manager, Wazuh Agents, ruleset customization, active responses, and SIEM integration.

2. **Elastic Documentation.**

Elasticsearch and Kibana Documentation.

Available at: <https://www.elastic.co/guide>

Description: Reference for Elasticsearch indexing, query mechanisms, and Kibana dashboard creation.

3. **Docker Documentation.**

Docker and Docker Compose.

Available at: <https://docs.docker.com>

Description: Useful for understanding containerization, Docker Compose configurations, networking, and orchestration of multi-container environments.

4. **Sysmon Documentation (Microsoft Sysinternals).**

System Monitor (Sysmon) Utility.

Available at: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>

Description: Comprehensive guide covering Sysmon configuration, event logs, and system monitoring capabilities.

8.2 Security Frameworks and Standards

5. MITRE ATT&CK Framework.

Adversarial Tactics, Techniques & Common Knowledge.

Available at: <https://attack.mitre.org>

Description: Framework used for threat mapping and correlating detection rules to attacker behavior.

6. NIST Special Publication 800-Series.

NIST Cybersecurity Framework.

Available at: <https://csrc.nist.gov>

Description: Guidelines that support security monitoring, incident response, and SOC structure.

8.3 Open-Source Tools & Community Contributions

7. SwiftOnSecurity Sysmon Configuration.

Sysmon Modular Configuration.

Available at: <https://github.com/SwiftOnSecurity/sysmon-config>

Description: Provides pre-defined Sysmon rules for enhanced Windows activity monitoring.

8. Wazuh Docker GitHub Repository.

Wazuh Docker Deployment Files.

Available at: <https://github.com/wazuh/wazuh-docker>

Description: Contains official Docker Compose files and configuration templates for Wazuh.

9. SecurityOnion Project.

SOC Tools and Threat Hunting Resources.

Available at: <https://securityonion.net>

Description: Offers broader SOC concepts, ELK integrations, and open-source security solutions.

8.4 Research Papers and Articles

10. Scarfone, K., & Mell, P. (2007).

Guide to Intrusion Detection and Prevention Systems (IDPS).

NIST Special Publication 800-94.

Description: A foundational resource for understanding SOC workflows, IDS/IPS systems, and threat detection models.

11. Stallings, W. (2018).

Computer Security: Principles and Practice. Pearson.

Description: Discusses cybersecurity principles, authentication, access control, and monitoring systems.

12. Kent, K., & Souppaya, M. (2006).

Guide to Computer Security Log Management.

NIST SP 800-92.

Description: Provides guidelines for collecting, analyzing, and storing security logs.

8.5 Cybersecurity Blogs and Learning Platforms

13. **Elastic Blog**

Security Analytics, Log Processing, and SIEM Features.

Available at: <https://www.elastic.co/blog>

14. **Wazuh Blog**

Use cases, ruleset examples, threat detection guides.

Available at: <https://wazuh.com/blog>

15. **Medium (Cybersecurity Publications)**

Articles on SOC operations, SIEM implementation, and detection engineering.

Available at: <https://medium.com>

8.6 Additional Useful References

16. OWASP Foundation.

Security Best Practices & Logging Guidelines.

Available at: <https://owasp.org>

17. TheHive Project Documentation.

Incident Response Platform & Case Management Concepts.

Available at: <https://docs.thehive-project.org>

18. Alcorn, M., & Savill, J. (2020).

Azure Security Monitoring and Logging. Microsoft Press.

Description: Discusses cloud-based logging and security monitoring concepts relevant to future scope.